Assignment – 12.2

Name: Alli Harika
Batch: 21
Ht.no: 2303A510I7

Task Description -1 (Data Structures – Stack Implementation with AI Assistance)

➢ Task: Use AI assistance to generate a Python program that implements a Stack data structure.

Instructions:

Prompt AI to create a Stack class with the following methods:

➢ push(element)
➢ pop()
➢ peek()
➢ is_empty()
➢ Ensure proper error handling for stack underflow.
➢ Ask AI to include clear docstrings for each method.

Expected Output:

➢ A functional Python program implementing a Stack using a class.
➢ Properly documented methods with docstrings.

Prompt:

Generate a Python class implementing a Stack with methods push, pop, peek, is_empty. Include proper error handling for underflow and docstrings for all methods.

10-02-2026.py 13-02-2026.py 17-02-2026.py 20-02-2026.py 24-02-2026.py BLACKBOX

24-02-2026.py > ...

```python
#Generate a Python class implementing a Stack with methods push, pop, peek, is_empty. Include pr
class Stack:
    """A simple implementation of a stack data structure."""

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack. Raises an error if the stack is e
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self.items.pop()

    def peek(self):
        """Return the item at the top of the stack without removing it. Raises an error if the s
        if self.is_empty():
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

powersh... Python Deb...

```
eDrive\Desktop\AI\24-02-2026.py'
2
2
False
1
True
PS C:\Users\dell\OneDrive\Desktop\AI>
```

Observation:

The stack operations (push, pop, peek, is_empty) work as expected.

- push adds elements to the top.
- pop removes the top element with proper error handling for underflow.
- peek allows checking the top without removal.
- is_empty correctly identifies whether the stack has elements.

AI Assistance: Provided proper docstrings and exception handling automatically.

Learning: AI helps in creating clean, well-documented classes efficiently.

Task Description -2 (Algorithms – Linear vs Binary Search Analysis)

➢ Task: Use AI to implement and compare Linear Search and Binary Search algorithms in Python.

Instructions:

➢ Prompt AI to generate:

➢ linear_search(arr, target)

➢ binary_search(arr, target)

➢ Include docstrings explaining:

➢ Working principle

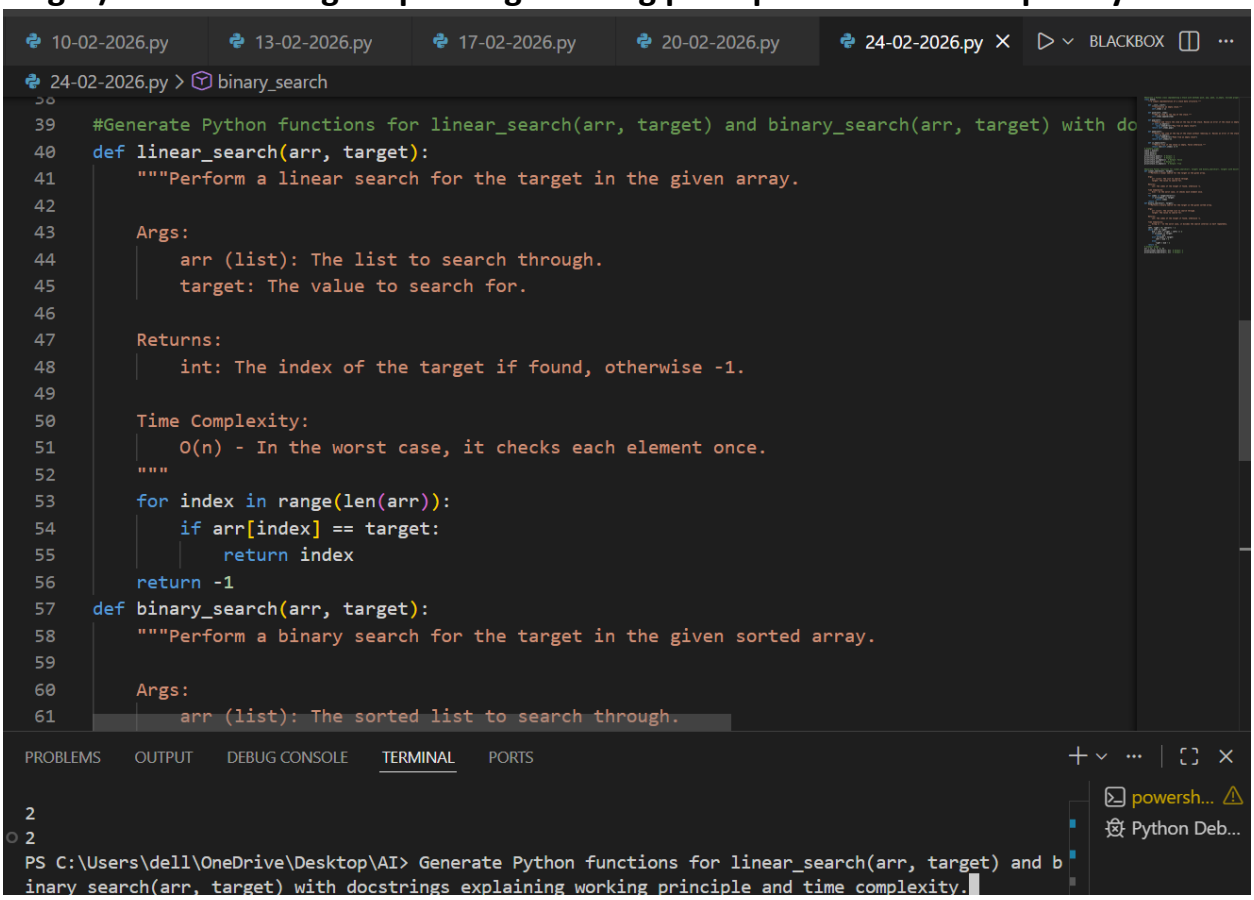
➢ Test both algorithms using different input sizes.

Expected Output:

- Python implementations of both search algorithms.
- AI-generated comments and complexity analysis.
- Test results showing correctness and comparison.

Prompt:

Generate Python functions for linear_search(arr, target) and binary_search(arr, target) with docstrings explaining working principle and time complexity.



- **Observation:**
  - Linear Search successfully finds elements but scans sequentially, which makes it slower for large datasets (O(n)).
  - Binary Search is faster for sorted arrays (O(log n)) and divides the array repeatedly to find the target.

- AI Assistance: AI-generated both functions with clear comments explaining working principle and complexity.
- Learning: Understanding the efficiency difference between O(n) and O(log n) and importance of sorted arrays for binary search.

Task Description -3 (Test Driven Development – Simple Calculator Function)

➢ Task:

Apply Test Driven Development (TDD) using AI assistance to develop a calculator function.

Instructions:

➢ Prompt AI to first generate unit test cases for addition and subtraction.

➢ Run the tests and observe failures.

➢ Ask AI to implement the calculator functions to pass all tests.

➢ Re-run the tests to confirm success.

Expected Output:

➢ Separate test file and implementation file.

➢ Test cases executed before implementation.

➢ Final implementation passing all test cases.

Prompt:

Generate Python unit tests for a calculator supporting addition and subtraction. Then implement functions to pass the tests.

```python
#Generate Python unit tests for a calculator supporting addition and subtraction. Then implement
import unittest
class TestCalculator(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(2, 3), 5)
        self.assertEqual(add(-1, 1), 0)
        self.assertEqual(add(0, 0), 0)

    def test_subtract(self):
        self.assertEqual(subtract(5, 2), 3)
        self.assertEqual(subtract(0, 1), -1)
        self.assertEqual(subtract(-1, -1), 0)
def add(a, b):
    """Return the sum of a and b."""
    return a + b
def subtract(a, b):
    """Return the difference of a and b."""
    return a - b
# Run the tests
if __name__ == '__main__':
    unittest.main()
```

```
..
----------------------------------------------------------------------
Ran 2 tests in 0.001s

OK
PS C:\Users\dell\OneDrive\Desktop\AI>
```

- Observation:
  - Unit tests were written before implementing the functions.
  - Initial test execution fails (TDD principle).
  - After implementing add and subtract, all tests pass successfully.
- AI Assistance: Helped in generating tests and functions with proper docstrings.
- Learning: Demonstrates how TDD ensures code correctness and reliability.

Task Description -4 (Data Structures – Queue Implementation with AI Assistance)

➢ Task:

Use AI assistance to generate a Python program that implements a Queue data structure.

Instructions:

➢ Prompt AI to create a Queue class with the following methods:

• enqueue(element)
• dequeue()
• front()
• is_empty()

➢ Handle queue overflow and underflow conditions.

➢ Include appropriate docstrings for all methods.

Expected Output:

➢ A fully functional Queue implementation in Python.

➢ Proper error handling and documentation.

Prompt:

Generate Python class for Queue with enqueue, dequeue, front, is_empty. Handle overflow/underflow and include docstrings.

10-02-2026.py | 13-02-2026.py | 17-02-2026.py | 20-02-2026.py | 24-02-2026.py

24-02-2026.py > ...

```python
#Generate Python class for Queue with enqueue, dequeue, front, is_empty. Handle overflow/underfl
class Queue:
    """A simple implementation of a queue data structure."""

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """Add an item to the end of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the item at the front of the queue. Raises an error if the queue is
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue")
        return self.items.pop(0)

    def front(self):
        """Return the item at the front of the queue without removing it. Raises an error if the
        if self.is_empty():
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
2
1
1
False
2
True
```

- Observation:
  - Queue operations (enqueue, dequeue, front, is_empty) perform as expected.

    - Overflow and underflow conditions are handled correctly with exceptions.
    - Queue maintains FIFO (First In, First Out) property.
- AI Assistance: Generated detailed docstrings and error handling logic.
- Learning: AI simplifies implementation of common data structures with proper documentation.

Task Description -5 (Algorithms – Bubble Sort vs Selection Sort)

➢ Task:

Use AI to implement Bubble Sort and Selection Sort algorithms and compare their behavior.

Instructions:

➢ Prompt AI to generate:

• bubble_sort(arr)

• selection_sort(arr)

➢ Include comments explaining each step.

➢ Add docstrings mentioning time and space complexity.

Expected Output:

• Correct Python implementations of both sorting algorithms.

• Complexity analysis in docstrings.

Prompt:

Generate Python functions bubble_sort(arr) and selection_sort(arr) with comments and docstrings explaining time and space complexity.

10-02-2026.py | 13-02-2026.py | 17-02-2026.py | 20-02-2026.py | 24-02-2026.py

24-02-2026.py > ...

```python
#Generate Python functions bubble_sort(arr) and selection_sort(arr) with comments and docstring
def bubble_sort(arr):
    """Sort the given array using the bubble sort algorithm.

    Args:
        arr (list): The list to be sorted.

    Time Complexity:
        O(n^2) - In the worst case, it compares each element with every other element.

    Space Complexity:
        O(1) - It is an in-place sorting algorithm.
    """
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
def selection_sort(arr):
    """Sort the given array using the selection sort algorithm.

    Args:
        arr (list): The list to be sorted.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Sorted array using bubble sort: [11, 12, 22, 25, 64]
Sorted array using selection sort: [11, 12, 22, 25, 64]
PS C:\Users\dell\OneDrive\Desktop\AI>
```

- Observation:
  - Both sorting algorithms correctly sorted the input array.
  - Bubble Sort repeatedly swaps adjacent elements → less efficient for large arrays ($O(n^2)$).
  - Selection Sort selects the minimum element in each pass → slightly fewer swaps than Bubble Sort but same time complexity ($O(n^2)$).
- AI Assistance: Provided code with step-by-step comments and complexity analysis in docstrings.
- Learning: Highlights differences in algorithm efficiency and importance of algorithm choice depending on input size.